An Agent-Based Approach to Automating Software Performance Testing (Work In Progress Paper)

Abstract

Performance testing is essential for contemporary software systems because it ensures reliability and user satisfaction (while keeping operational costs under control). Conventional performance testing activities still require expert knowledge, laborious scripting, and substantial time investment within modern DevOps workflows. Recent advances in generative and agentic AI create the opportunity to automate these processes. This work-in-progress paper explores the use of agentic AI to support end-to-end automation of software performance testing. We present an initial prototype that coordinates multiple specialized agents to extract contextual information from existing project artifacts, generate realistic performance test scenarios, execute tests using an established performance-testing framework, and provide structured interpretations of the resulting metrics. We report the early results based on exploratory case studies of microservice-based systems from the DeathStarBench benchmark suite, which suggests that agentic workflows can autonomously produce executable performance tests and meaningful performance reports with limited human intervention, even when available context is incomplete or partially misleading. Such results illustrate the feasibility of agent-based performance testing and motivate a broader research agenda on AI-augmented performance engineering.

CCS Concepts

• **General and reference** → **Measurement**; **Evaluation**; • **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → **Software performance**.

Keywords

Performance, Testing, Microservice, Agentic AI, Load testing

ACM Reference Format:

. 2018. An Agent-Based Approach to Automating Software Performance Testing (Work In Progress Paper). In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (ICPE '26).* ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Software performance testing is an essential part of software quality assurance because it ensures that a system meets performance requirements and meets user expectations [6]. As modern applications grow in complexity, performance directly affects operational efficiency and business continuity (slow or unstable systems quickly turn into dissatisfied users, financial loss, and reputation damage). Despite decades of incremental automation, performance testing still demands considerable manual work, specialized knowledge, and long lead times, so it frequently becomes a bottleneck in continuous integration and delivery pipelines.

Performance testing refers to the process of measuring and evaluating the performance-related aspects of a system under test (for example, response time, resource usage, or throughput) [9]. The process can target the overall system or concentrate on a particular subsystem [26]. The term is often used interchangeably with load testing [10], although load testing focuses specifically on evaluating behavior under defined levels of stress so that load-related issues can be identified [8]. Traditionally, performance testing relies on a complementary toolkit of measurement, simulation, analytical modeling, and experimental design [20]. Those approaches are largely manual and depend on expert judgment to select configurations, prepare scripts, and define thresholds (an effort that is both time-consuming and difficult to adapt to changing requirements).

The rise of microservices and cloud-native architectures has introduced additional complexity because inter-service dependencies and dynamic scaling behaviors create constantly shifting performance profiles [17]. These characteristics demand adaptive and intelligent testing frameworks capable of reacting to runtime changes in real time, an area where recent advances in AI technologies can contribute. Mascia et al. present a learning-based framework that generates critical performance workloads for microservices automatically [25]. Ramamoorthi proposes a framework that integrates AI-driven monitoring and predictive analytics so that resource management becomes dynamic during performance testing [27]. Nevertheless, the application of generative and agentic AI to end-to-end automation of performance testing remains largely unexplored.

This work-in-progress paper explores whether and how agentic AI could support the automation of software performance testing with the long-term goal of reducing manual effort and preserving test relevance. To such an end, we present an early-stage prototype that orchestrates multiple specialized agents to support performance testing tasks and investigate the following research question: **To what extent can agentic AI support autonomous generation, execution, and interpretation of software performance tests with minimal human intervention?**

We adopt a proof-of-concept approach by developing an early-stage prototype that demonstrates the practical application of agentic AI and agent-to-agent workflows for performance-testing automation. The prototype integrates LangGraph [23] with the Model Context Protocol (MCP) [19] and the PPTAM performance-testing framework [7]. It extracts information from the system codebase, generates performance test scenarios, executes them via PPTAM, and analyzes results through LLM-based reasoning. We adopt an

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICPE '26, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

exploratory evaluation approach using two microservice-oriented benchmarking applications from the *DeathStarBench* benchmark suite [18], thus demonstrating the feasibility and potential benefits of AI-driven automation in realistic large-scale environments. Through such experiments, we aim to understand how agentic approaches are effective, where they fail, and which aspects remain difficult to automate.

The remainder of the paper is organized as follows. Section 2 surveys related work. Section 3 introduces the proposed framework and describes the prototype implementation. Section 4 presents an exploratory evaluation using two benchmark systems. Section 5 discusses the insights, limitation and broader implications for future research, and Sect. 6 concludes the paper.

2 Related work

Software performance testing has traditionally been a systematic process of selecting techniques, metrics, and workloads that match specific systems (no universal method fits every architecture) [20]. The seminal work by Jain formalizes a process that identifies metrics, defines workloads, controls experiments, and applies analytical models so that insights remain meaningful, replicable, and context-aware [20]. Weyuker and Vokolos extend that structure by centering the process on disciplined workload design and architecture-based test construction rather than ad hoc benchmarks [31]. Additional workload-focused approaches, such as the one by Avritzer et al., emphasize realistic modeling derived from system characterization and iterative validation [6]. Denaro et al. advocate early performance testing driven by architecture knowledge and reusable middleware components [16]. All of these contributions highlight the importance of rigor but they also expose how manual configuration and limited adaptivity constrain traditional performance testing.

Automation research seeks to reduce that burden. Jovic et al. record and replay GUI-based user interactions for Java applications so that perceptible performance can be measured repeatedly [22]. Javed et al. embed an automated toolchain in Continuous Integration pipelines (task specification via YAML, runtime monitoring using VyPR, and data analysis that feeds actionable insights) [21]. Yang et al. provide an automated capability for evaluating and optimizing deep-learning deployments across GPUs by generating experiments, collecting metrics, and producing visual analysis reports [32]. These advances improve efficiency, yet most remain tied to narrow domains and static configurations, so they still lack the generality and adaptivity needed for contemporary systems.

The emergence of microservice architectures prompted targeted performance-testing research. Eismann et al. empirically show how some benefits of microservices introduce notable performance costs, complicating the quest for consistent and replicable testing [17]. Cooper et al. develop a trace-based, budget-aware mechanism that prioritizes short but representative performance tests suited for CI/CD environments [13]. Lei et al. employ the Kubemark simulation framework to provide Kubernetes-based performance and scalability testing, enabling large-scale yet cost-effective benchmarking of microservice deployments [24].

Automation tailored to microservices continues this trajectory. Camilli et al. present a DevOps-integrated framework that links performance testing with reliability assessment so that correlations between the two can be detected [12]. De Camargo et al. propose a decentralized, service-embedded solution that automatically discovers and executes test specifications for each microservice [15]. Avritzer et al. deliver a domain-driven and quantitative framework that evaluates alternative deployment configurations under realistic workloads by combining operational profiles, automated load testing, and scalability analysis [5]. Despite these innovations, most methods remain rule-based, static, and tightly coupled to specific architectures, which limits on one hand, their ability to adapt to evolving microservice ecosystems in real time, and on the other hand hinders approaches as continuous autonomous testing in dynamic, Lean environments (see e.g., [14]).

The rapid maturation of AI has also elevated interest in agentic approaches within software engineering [1]. Although recent studies outline potential uses of agentic AI throughout the software development life cycle [3], we are not aware of any prior work that couples agent-based reasoning with established performance-testing frameworks.

3 Framework Design

This section details the architectural, conceptual, and technical design of the proposed agentic-AI framework for automating software performance testing. The objective is to deliver an agent-driven desktop application that autonomously generates scenarios, prepares test environments, executes load tests, and analyzes results by coordinating a suite of specialized agents obtaining data from sources using MCP.

3.1 Overall Architecture

The overall architecture integrates agentic AI, structured context retrieval, and automated test execution into a cohesive, modular framework. It is organized into three complementary perspectives (system context, container architecture, and the internal structure of the LangGraph agent system). Collectively, these layers show how the Performance Testing Agent orchestrates every component required for fully automated scenario generation, environment configuration, execution, and result interpretation.

Shown in Figure 1, the overall architecture of the performance testing agent includes four core components that collaborate to support end-to-end automation. A tester-facing Tauri [29] desktop application (Svelte [28] frontend plus Rust [30] backend) orchestrates the workflow by collecting input, rendering agent outputs, coordinating LangGraph calls, launching sidecar processes, managing persistence, and issuing PPTAM execution requests. The PPTAM Runner is a containerized Python service that wraps PPTAM's APIs with a Flask REST interface, replacing configuration files with dynamic POST payloads. A bundled LangGraph Agent, exposed through an *AIOHTTP* server [2], integrates MCP Servers (Codebase, Swagger, GitHub, Atlassian) and hosts nine specialized agents (*Project Purpose*, *Project Insights*, *Project Features*, *Feature Scenarios*, *Environment Setup*, *Performance Test* (Locust generation), *PPTAM Wrapper*, *Result Interpreter*) so the desktop application can trigger each capability independently while building on prior outputs. Finally, a SQLite persistence layer stores the software-under-test (SUT) metadata, generated scenarios, PPTAM configurations, and

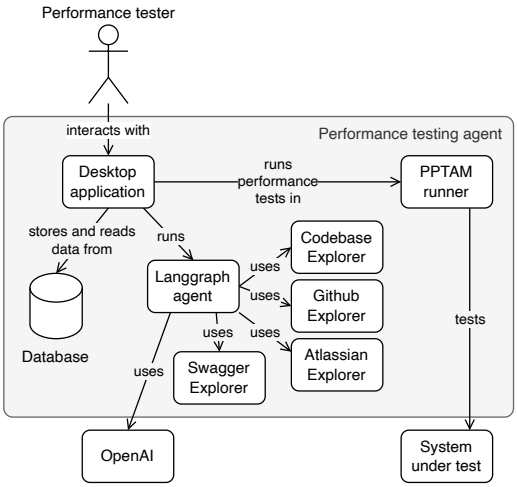

Figure 1: Container diagram of the overall architecture, depicted using the C4 model notation [11].

raw as well as interpreted results so that runs can be resumed or compared without recomputation.

To provide reliable perspectives on the system under test, we integrate four MCP-based third-party resources that complement the LangGraph agents. The Codebase Explorer supplies read-only access to source files, directory structures, and search capabilities so agents can confidently extract architecture, authentication, endpoint, and performance insights. GitHub Explorer contributes repository metadata (issues, milestones, pull requests, documentation) and a historical view of project health, helping agents rank critical paths. Atlassian Explorer exposes Jira and Confluence artifacts so that generated tests align with documented business value, dependencies, and acceptance criteria. Finally, the Swagger Explorer delivers machine-readable API specifications (for example Swagger/OpenAPI) that detail endpoints, schemas, and authentication rules, which is vital for accurate *Locust* plan generation and authenticated workload synthesis.

The custom Codebase Explorer exposes only the operations the agents consistently relied on during our evaluations (for example `list_files_in_directory`, `get_file_content`, or `find_files_with_content` for regex-based searches across large microservice trees). By constraining the surface area we prevent LLMs from getting distracted by tangential tooling and we make it feasible to run scoped, iterative reasoning loops even when repositories contain hundreds of services.

Together, these third-party resources enable agents to build a holistic, multidimensional understanding of the SUT, supporting a more holistic, multidimensional understanding of the SUT, which may improve the relevance of automated performance tests.

3.2 Agent Architecture

The framework relies on a modular, multi-agent architecture implemented with LangGraph. Each agent is modeled as a node in a graph and follows an adapted version of the *ReAct* paradigm, in which reasoning steps are interleaved with tool invocations. We extend ReAct [33] with agent-specific state structures, long-term memory utilities, and knowledge-shelf interfaces so that the agents retain context across iterations, store intermediate artifacts, and progressively refine their outputs without exceeding model-context limits. Tool invocation is enabled through the connected MCP Servers, which provide structured access to the SUT's codebase, documentation, issue trackers, and API definitions. This design empowers the agents to explore the SUT autonomously, plan multi-step analyses, and branch execution paths if they discover new information.

The ReAct architecture combines two fundamental capabilities: reasoning (Thought) and acting (Tool use). An agent alternates between generating internal reasoning steps and calling external tools to query data, test hypotheses, or perform computations. This loop enables the agent to adapt to new evidence, reconsider earlier assumptions, and make well-informed decisions instead of delivering a single static answer. The architecture typically comprises the nodes START, Thinking, Tool, and END, which together support stateful decision-making. ReAct is particularly effective for tasks that require both cognitive planning and interaction with artifacts such as codebases or documentation.

All nine agents in the system share this modified ReAct foundation so they can operate autonomously and iteratively while following a unified reasoning-and-action cycle. Each agent balances internal reasoning with MCP-powered tool invocations to gather evidence from source code, documentation, issue trackers, and API specifications. The shared framework allows the agents to maintain and update their internal state using various plug-ins (for example short- and long-term memory tools or debugging facilities) so that high-quality outputs can be produced in a single modeling sequence. By embedding every agent within this common cognitive structure, the system keeps feature extraction, environment preparation, Locust generation, and result interpretation both adaptable to the SUT and coherent throughout the performance-testing workflow.

The nine agents cover the entire workflow from contextual understanding to result interpretation. The *Project Purpose* agent produces a high-level description of the SUT's intent, domain, and user value proposition, repeatedly querying MCP resources until the summary stabilizes so later agents inherit a consistent brief. *Project Insights* inspects technical characteristics such as languages, frameworks, storage tiers, and authentication mechanisms; it records concrete endpoints (for example OAuth or JWT flows), highlights third-party services, and captures architectural styles to anchor downstream reasoning. *Project Features* catalogs domain capabilities with names, descriptions, and importance levels while linking back to evidence in code or issues, which enables scenario targeting without re-parsing the entire repository. *Feature Difficulties* evaluates implementation challenges by correlating code paths with contextual artifacts (issues, comments) and explicitly surfaces hypothesized bottlenecks such as fan-out queries or synchronous RPC chains. Building on that, *Feature Scenarios* generates realistic user interaction flows that include preconditions, user actions, and system responses; it revisits the set to ensure negative and edge cases are covered before finalization. *Project Environment* infers how to prepare a runnable environment by synthesizing Dockerfiles, docker-compose manifests, and endpoint metadata while noting service dependencies, required environment variables, and the base URL. *Performance Test* converts the selected scenarios into

executable Locust workloads, calling live endpoints through debugging hooks to validate payloads, authentication, and cookies before embedding them in task definitions. *PPTAM Wrapper* then translates the Locust configuration and chosen test objective (load, stress, endurance, etc.) into a structured PPTAM plan, sequencing warm-up, steady-state, and ramp-down steps and annotating spawn rates plus user counts for reusability. Finally, *Result Interpreter* analyzes PPTAM metrics (response percentiles, throughput, failures), cross-references them with operational profiles, and produces developer-friendly findings and recommendations, flagging whether headroom meets expectations and suggesting follow-up investigations when anomalies appear.

3.3 Analysis Process

With the architecture in place, the analysis process unfolds as a multi-stage, AI-driven workflow in which the LangGraph agents progressively deepen their understanding of the SUT by integrating evidence from code, documentation, and project artifacts. The process begins with high-level contextualization: the *Project Purpose* agent characterizes the system's intent, and the *Project Insights* agent determines the technology stack and authentication mechanisms. Next, the *Project Features*, *Feature Difficulties*, and *Feature Scenarios* agents build a domain-specific model by cataloging available functionality, assessing implementation complexity, and formulating realistic user interaction flows. All of these steps leverage the Codebase, GitHub, Atlassian, and Swagger explorers to ensure that reasoning is grounded in trustworthy evidence.

The *Project Environment* agent then interprets the deployment structure of the SUT, prepares a runnable environment (typically through containerization), and determines how the application will be accessed. Once the environment is known, the *Performance Test* agent synthesizes a Locust workload tailored to the selected feature and test goal, validating behaviors through live API calls when necessary. The *PPTAM Wrapper* agent transforms the resulting Locust configuration and test parameters into a PPTAM plan executed by the PPTAM Runner. Finally, the *Result Interpreter* agent reviews the raw performance metrics, relates them to the operational profile and previously derived insights, and produces a report.

Overall, the process is a layered pipeline in which each agent refines and expands the shared understanding of the SUT. By combining iterative reasoning, disciplined tool use, and multi-source evidence, the framework automates the traditionally manual stages of system comprehension, test design, environment preparation, and result interpretation, creating an integrated end-to-end workflow for performance testing.

3.4 Scenario Design Workflow

Early experimentation revealed that agentic LLMs work best when the scenario-derivation process is explicitly staged, so the workflow mirrors how experienced performance engineers plan tests: first, the Project Purpose agent frames the project by synthesizing a reusable brief; second, Project Insights performs technical reconnaissance to inventory languages, frameworks, authentication flows, and stateful dependencies; third, Project Features enumerates and ranks user-facing capabilities so workloads target meaningful behavior; fourth, Feature Difficulties conducts bottleneck brainstorming by correlating architecture data with historical cues to surface likely contention points; fifth, Feature Scenarios authors executable journeys and validates endpoints through debugging hooks; and finally, the Performance Test and PPTAM Wrapper agents package execution by emitting Locust scripts and PPTAM plans that can be run immediately. This structure made it possible to regenerate tests quickly during the case studies while keeping the agents focused on one decision at a time.

4 Exploratory Evaluation

This section describes the exploratory evaluation of the proposed agent-based automated software performance testing solution. Herein, we adopt and execute two open-source real-world applications in order to evaluate the feasibility and effectiveness of the approach.

Regarding the *Evaluation Objectives*, we focus on how agents reason, where they fail, and how much manual intervention is required to recover from errors. This perspective reflects the exploratory nature of the work and supports the identification of open challenges for future research. Hence, the exploratory evaluation is guided by three high-level objectives: 1) *Feasibility*: to assess whether the agent-based framework can autonomously progress through the major stages of performance testing, including system comprehension, scenario derivation, test execution, and result interpretation; 2) *Behavioral Plausibility*: to examine whether the generated artifacts (e.g., scenarios, workloads, configurations, and reports) align with the structure and intent of the system under test; 3) *Robustness Under Imperfect Context*: to observe how agents behave when confronted with incomplete, outdated, or misleading project artifacts, which are common in real-world settings.

To ensure that the empirical evaluation reflects realistic and diverse contexts, we selected two open-source microservice applications from the *DeathStarBench* benchmark suite [18]. DeathStarBench provides production-grade cloud-native workloads that are widely used in performance engineering research and are well suited for studying distributed system behavior.

Applying these criteria led to the choice of the *Hotel Reservation* and *Social Network* applications. Hotel Reservation is a medium-scale system composed of ten microservices that simulate a hotel reservation workflow (including authentication, availability checks, recommendations, and payment). It serves as a baseline for evaluating the framework in a moderately complex context. Social Network is a large-scale platform that emulates a social media ecosystem with capabilities for user management, content handling, and interaction, including thrift-based inter-service contracts. This system stresses scalability and robustness. The contrast between the two applications enables a comprehensive comparison of how the framework adapts to different microservice architectures.

4.1 Experiment Setup and Execution

Both experiments were designed to maintain methodological consistency. Each execution took place in a controlled environment to minimize extraneous variability, and the framework operated under identical configurations with only the *Codebase Explorer MCP Server* enabled. Restricting context to static code analysis was an intentional choice so we could examine the agents' ability to perform

autonomous reasoning without support from API documentation, issue trackers, or project-management data.

Neither application includes Swagger endpoints (for example swagger.json or OpenAPI files) or Atlassian tooling (such as Jira or Confluence), so Codebase Explorer was the sole structured context source. In addition, all Docker-related files were removed to test whether the agents could reconstruct them using only available documentation and source code.

Faults were injected modifying contextual artifacts rather than by changing the systems' operational logic. In Hotel Reservation, misleading data appeared as a fabricated README feature and a decoy service directory, testing the agents' ability to detect inconsistencies between documentation and code. In Social Network, altered documentation and extra service-like folders mimicked outdated project structures. These modifications emulate the incomplete, contradictory, or deceptive information commonly encountered in industrial environments and allow to assess robustness.

After setup, the tool processes each application independently while faulty artifacts remain untouched so we can observe the natural response. Each run therefore proceeds through a fixed sequence—Project Purpose analysis, Project Insights extraction, Feature Difficulty assessment, Scenario generation, Environment configuration and containerization, Performance Test construction, Test execution, and Result interpretation and reporting—ensuring the full pipeline is exercised every time.

Although both applications follow the same workflow, their architectural structures and documentation profiles introduce notable differences. In Hotel Reservation, agents rely heavily on distributed service directories and REST endpoints, which leads to deeper file traversal and granular inspection of service-specific logic. The injected fabrications tricked some agents into referencing nonexistent behavior and produced incorrect Docker configurations, exposing limits in cross-validation and environment inference. Social Network centralizes much of its architecture in the *social_network.thrift* file, allowing agents to capture intent quickly but forcing them to depend more on interface specifications than on dispersed code. Scenario generation also diverged: Hotel Reservation focused on multi-service search workflows, whereas Social Network prioritized post-creation flows that demanded authentication and cookie handling. These observations highlight how repository structure, communication patterns, and documentation formats shape reasoning paths, environment synthesis, and the resulting performance scenarios.

4.2 Preliminary Results

The desktop application guides practitioners through four main views when preparing the Hotel Reservation system. The Project Insights screen summarizes the technology stack and the authentication mechanisms extracted from the codebase. The Feature Difficulties view highlights potential bottlenecks per feature (such as cache calls or downstream RPC chains) and ranks them by perceived risk. The Project Features catalog lists each customer-facing capability together with its user value and importance, making it easy to select a target for performance testing. Finally, the PPTAM configuration view allows testers to inspect or tweak the automatically generated load profiles (for example, ramp-up duration, spawn rate, or user count) before execution. Together, these panels provide transparency into each agent's contribution and let testers intervene when necessary without writing scripts.

Both systems indicate that the framework was able to run end-to-end performance tests for microservice applications while presenting structured, summarized results. In the Hotel Reservation scenario, the agents autonomously generated and executed the full test suite. They navigated the codebase accurately, extracted relevant features, and built realistic scenarios, including edge cases such as searches with no availability or flexible dates. The Feature Scenarios agent explicitly revisited all candidate journeys before finalizing the Locust blueprint, and the tester selected *Load Testing* as the initial objective. The *Project Environment* agent attempted to compose Dockerfiles for each Go microservice but failed because it assumed the presence of `go.mod`/`go.sum`. Once the original DeathStarBench Docker artifacts were restored, the *Performance Test* and *PPTAM Wrapper* agents executed without manual scripting. The *Result Interpreter* agent mapped the collected metrics to the operational profile and reported median response times of 7–8 ms, 90th percentile below 12 ms, a maximum of 26 ms, an aggregate throughput of 1.5 requests/second, and zero failures, thereby suggesting that the search feature comfortably exceeds its 300-users-per-day target.

The Social Network scenario offered a larger, more interconnected service landscape. Despite the deliberate removal of documentation, the *Project Purpose* and *Project Insights* agents reconstructed the system's goals and JWT-based authentication scheme directly from the thrift definitions. The *Project Features* agent missed documentation-only functionality (for example, timeline ranking), illustrating the benefit of multi-source context. Containerization remained problematic because the generated configuration omitted the Nginx proxy. Nevertheless, the *Performance Test* agent encoded login flows, media uploads, and mentions inside the Locust script, and the *PPTAM Wrapper* orchestrated a load test over `/api/post/compose`. The resulting report covered 746 requests (2.5 requests/second) with a median response of 1.1 s, 90th percentile of 1.8 s, 99th percentile of 2.3 s, a maximum of 2.9 s, and zero failures, all within the 5 s SLA for interactive posting.

These exploratory observations suggest that agent-based workflows can support key stages of performance testing with limited human intervention. In particular, scenario derivation and workload synthesis benefited from agentic reasoning over code structure and inferred system intent. At the same time, the evaluation exposed clear limitations. Agents occasionally over-trusted misleading documentation and struggled with environment configuration when expected artifacts were missing. These issues did not prevent end-to-end execution but required manual correction or reuse of existing configurations. The experiments also revealed that implicit or emergent system features are difficult for agents to detect without explicit documentation or runtime signals.

Overall, the evaluation highlights both the promise and current limitations of agent-based performance testing. Rather than providing definitive evidence of feasibility, the results serve to characterize system behavior, identify failure modes, and motivate future research directions, which aligns with the goals of a work-in-progress study.

The developed source code as well as screen shots of the developed user interface are available at [4].

5 Insights, Limitation and Future Agenda

The preliminary results suggest that agentic reasoning can support end-to-end performance testing workflows without manual script authoring. Across both subject systems, agents were able to derive performance-relevant scenarios, synthesize executable workloads, and produce structured reports by reasoning over system artifacts. This indicates that performance testing tasks traditionally performed by human experts can, at least in part, be decomposed into agent-level responsibilities. Meanwhile, scenario derivation benefited most from agentic reasoning. By correlating code structure, inferred system intent, and architectural cues, agents were able to generate realistic user journeys, including negative and edge cases. This aligns with the way experienced performance engineers reason about workload relevance and suggests that agentic approaches may be particularly effective in early-stage test design. Furthermore, the evaluation revealed that environment preparation remains a critical bottleneck. When expected infrastructure or build artifacts were missing, agents struggled to infer complete execution environments. Although such failures were surfaced early and did not prevent further experimentation, they highlight the limits of static reasoning and the need for stronger environment-synthesis strategies.

The exploratory nature of this study imposes several limitations. From a technical perspective, agents occasionally over-relied on documentation artifacts without sufficiently validating them against source code, leading to incorrect assumptions about feature availability. Similarly, secondary or implicit functionality remained difficult to detect when not explicitly documented, reducing scenario diversity. From an empirical perspective, the evaluation is limited to two microservice-based benchmark systems and single executions per scenario. The absence of repeated runs, baseline comparisons, and practitioner studies limits the generalizability of the findings. In addition, generated artifacts (e.g., Locust scripts or Docker configurations) serve as proxies for reasoning quality and do not fully capture the correctness or completeness of agent decision-making. These limitations are intentional in the context of a work-in-progress study, as the goal is to highlight challenges and research questions rather than provide definitive evidence of feasibility.

This work represents an initial step toward agent-based automation of software performance testing. While the prototype demonstrates feasibility, it also exposes several open research challenges that must be addressed before agentic performance testing can be considered reliable, general, and broadly applicable. We outline a research agenda along four key dimensions.

- **Validation of Agent-Generated Performance Artifacts.** A central challenge concerns how to assess the correctness and adequacy of agent-generated artifacts such as performance scenarios, workload models, and test configurations. Unlike traditional scripts authored by human experts, agent-generated tests may be plausible, yet incomplete, or misaligned with true operational profiles. Future work should investigate systematic validation strategies, including cross-checking generated workloads against runtime traces, architectural models, or historical production data.
- **Robustness to Incomplete, Inconsistent, or Misleading Context.** Our experiments show that agents can operate with limited context but may over-trust documentation artifacts that diverge from the implementation. Strengthening cross-validation across heterogeneous sources (code, configuration files, live endpoints, and observability signals) remains an open problem. Research is needed on agent-level uncertainty estimation, contradiction detection, and self-critique mechanisms that allow agents to explicitly reason about confidence and risk.
- **Human–Agent Collaboration Models.** Fully autonomous performance testing may not always be desirable in practice. An important direction is the design of mixed-initiative workflows in which human performance engineers guide, validate, or override agent decisions at critical stages (e.g., workload selection, SLA interpretation, or environment setup). Understanding where human expertise provides the highest leverage is an open research question.
- **Generalizability and Empirical Grounding.** The current evaluation is limited to two microservice-based benchmark systems and single executions per scenario. Future studies should broaden the empirical basis by including additional architectural styles (e.g., monoliths or hybrid systems), repeated executions to assess stability, and comparative baselines against manually authored performance tests. Such studies are necessary to understand when and why agentic approaches outperform, match, or fall short of traditional practices.

Addressing these challenges will be critical to transform agent-based performance testing from a promising prototype into a reliable component of performance engineering tool chains. We view this work as an invitation for the community to explore agentic AI not only as an automation mechanism, but as a new paradigm for reasoning about performance in complex software systems.

6 Conclusions and Future work

This work-in-progress paper explored the feasibility of applying agentic AI to automate key stages of software performance testing. We presented an early-stage prototype that integrates multi-agent reasoning with structured context retrieval and an established performance testing framework, and we exercised it on representative microservice-based systems. Rather than aiming for comprehensive validation, the study was designed to probe whether agentic workflows can plausibly support system understanding, scenario derivation, test execution, and result interpretation with limited human intervention. The exploratory evaluation demonstrates that such workflows are technically viable and can operate end-to-end, while also revealing important challenges that motivate further research. Overall, this work contributes an initial empirical and conceptual foundation for agent-based performance testing and positions agentic AI as a promising direction for rethinking how performance engineering tasks may be approached in increasingly complex software systems.

References

[1] Deepak Bhaskar Acharya, Karthigeyan Kuppan, and B Divya. 2025. Agentic ai: Autonomous intelligence for complex goals–a comprehensive survey. *IEEe Access* (2025).

[2] aiohttp Contributors. 2026. aiohttp Documentation (stable). https://docs.aiohttp.org/en/stable/. Accessed: 2026-01-22.

[3] Muhammad Azeem Akbar, Arif Ali Khan, Muhammad Hamza, Abdullah Ghaffar, and Arash Hajikhani. 2025. Agentic AI in Software Engineering: Practitioner Perspectives Across the Software Development Life Cycle. *Software Engineering: Practitioner Perspectives Across the Software Development Life Cycle (September 16, 2025)* (2025).

[4] Anonymous Anonymous. 2026. Replication package for the paper "An Agent-Based Approach to Automating Software Performance Testing (Work In Progress Paper)". https://doi.org/10.5281/zenodo.18344041. doi:10.5281/zenodo.18344041

[5] Alberto Avritzer, Vincenzo Ferme, Andrea Janes, Barbara Russo, Henning Schulz, and André van Hoorn. 2018. A quantitative approach for the assessment of microservice architecture deployment alternatives by automated performance testing. In *European Conference on Software Architecture*. Springer, 159–174.

[6] Alberto Avritzer, Joe Kondek, Danielle Liu, and Elaine J Weyuker. 2002. Software performance testing based on workload characterization. In *Proceedings of the 3rd International Workshop on Software and Performance*. ACM, Rome, Italy, 17–24.

[7] Alberto Avritzer, Daniel Menasché, Vilc Rufino, Barbara Russo, Andrea Janes, Vincenzo Ferme, André van Hoorn, and Henning Schulz. 2019. PPTAM: production and performance testing based application monitoring. In *Companion of the 2019 ACM/SPEC International Conference on Performance Engineering*. 39–40.

[8] Cornel Barna, Marin Litoiu, and Hamoun Ghanbari. 2011. Autonomic load-testing framework. In *Proceedings of the 8th ACM international conference on Autonomic computing*. 91–100.

[9] Cornel Barna, Marin Litoiu, and Hamoun Ghanbari. 2011. Model-based performance testing (NIER track). In *Proceedings of the 33rd International Conference on Software Engineering*. 872–875.

[10] Mohamad Bayan and João W Cangussu. 2008. Automatic feedback, control-based, stress and load testing. In *Proceedings of the 2008 ACM symposium on Applied computing*. 661–666.

[11] Simon Brown. 2025. C4 model for visualising software architecture. https://c4model.com. Official website for the C4 model.

[12] Matteo Camilli, Antonio Guerriero, Andrea Janes, Barbara Russo, and Stefano Russo. 2022. Microservices integrated performance and reliability testing. In *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test*. 29–39.

[13] Quinn Cooper, Diwakar Krishnamurthy, and Yasaman Amannejad. 2024. Budget aware performance test selection for microservices. In *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*. IEEE, 376–385.

[14] Emanuele Danovaro, Andrea Janes, and Giancarlo Succi. 2008. Jidoka in software development. In *Companion to the 23rd ACM SIGPLAN Conference on Object-Oriented Programming Systems Languages and Applications* (Nashville, TN, USA) *(OOPSLA Companion '08)*. Association for Computing Machinery, New York, NY, USA, 827–830. doi:10.1145/1449814.1449874

[15] André De Camargo, Ivan Salvadori, Ronaldo dos Santos Mello, and Frank Siqueira. 2016. An architecture to automate performance tests on microservices. In *Proceedings of the 18th international conference on information integration and web-based applications and services*. 422–429.

[16] Giovanni Denaro, Andrea Polini, and Wolfgang Emmerich. 2004. Early performance testing of distributed software applications. In *Proceedings of the 4th International Workshop on Software and Performance*. 94–103.

[17] Simon Eismann, Cor-Paul Bezemer, Weiyi Shang, Dušan Okanović, and André Van Hoorn. 2020. Microservices: A performance tester's dream or nightmare?. In *Proceedings of the ACM/SPEC international conference on performance engineering*. 138–149.

[18] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 3–18.

[19] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278* (2025).

[20] Raj Jain. 1990. *The Art of Computer Systems Performance Analysis*. John Wiley & Sons.

[21] Omar Javed, Joshua Heneage Dawes, Marta Han, Giovanni Franzoni, Andreas Pfeiffer, Giles Reger, and Walter Binder. 2020. Perfci: A toolchain for automated performance testing during continuous integration of python projects. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 1344–1348.

[22] Milan Jovic, Andrea Adamoli, Dmitrijs Zaparanuks, and Matthias Hauswirth. 2010. Automating performance testing of interactive java applications. In *Proceedings of the 5th Workshop on Automation of Software Test*. 8–15.

[23] LangChain Developers. 2026. LangGraph. https://www.langchain.com/langgraph. Accessed: 2026-01-22.

[24] Qing Lei, Weidong Liao, Yingtao Jiang, Mei Yang, and Haifeng Li. 2019. Performance and scalability testing strategy based on kubemark. In *2019 IEEE 4th International Conference on Cloud Computing and Big Data Analysis (ICCCBDA)*. IEEE, 511–516.

[25] Cristian Mascia, Luca Giamattei, Antonio Guerriero, Roberto Pietrantuono, and Stefano Russo. 2025. Learning-based Automated Generation of Critical Workload Configurations for Microservices Performance Testing. In *2025 IEEE International Conference on Web Services (ICWS)*. IEEE, 764–774.

[26] BA Pozin and Igor V Galakhov. 2011. Models in performance testing. *Programming and Computer Software* 37, 1 (2011), 15–25.

[27] Vijay Ramamoorthi et al. 2024. AI-Enhanced Performance Optimization for Microservice-Based Systems. *Journal of Advanced Computing Systems* 4, 9 (2024), 1–7.

[28] Svelte Contributors. 2026. Svelte. https://svelte.dev/. Accessed: 2026-01-22.

[29] Tauri Contributors. 2026. Tauri v2. https://v2.tauri.app/. Accessed: 2026-01-22.

[30] The Rust Project Developers. 2026. Rust Programming Language. https://rust-lang.org/. Accessed: 2026-01-22.

[31] Elaine J Weyuker and Filippos I Vokolos. 2002. Experience with performance testing of software systems: issues, an approach, and case study. *IEEE transactions on software engineering* 26, 12 (2002), 1147–1156.

[32] Zhe Yang, Wei He, Fenglong Cai, Yonghui Xu, Wei Guo, and Lizhen Cui. 2024. AutoMP: A Tool to Automate Performance Testing for Model Placement on GPUs. In *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*. IEEE, 1599–1606.

[33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* (2022).